Zephyr Project

IT-Schulungen.com
Technology & Transformation

# Sensor API, Bus und Emulator

Verarbeitungskette, API-Walkthrough und Verständnisfragen

4.03

# Vom Devicetree bis zum Messwert

**SENSORPFAD**

```
Devicetree → Bus → Treiber → Sensor API → Anwendung
```

| | |
|---|---|
| **Devicetree** | beschreibt Bus, Gerät, compatible und Adresse |
| **Buscontroller** | transportiert Zugriffe zwischen Treiber und Gerät |
| **Sensor + Emulator** | liefert Kanäle und kontrollierte Stimuli |
| **Sensortreiber** | bindet das Gerät und implementiert die Sensor API |
| **Anwendung** | nutzt Device, Channels und Rückgabewerte |

Jede Ebene hat eine eigene Verantwortung – die Anwendung bleibt treiberunabhängig.

# Geräteinstanz aus dem Devicetree

## Referenz und Gerätebereitschaft vor dem ersten Zugriff

| 01 | 02 | 03 | 04 | 05 |
|---|---|---|---|---|
| **Alias** | **Knoten** | **Device** | **Ready** | **Anwendung** |
| motion-sensor | DT_ALIAS( motion_sensor) | DEVICE_DT_GET(...) | device_is_ready(dev) | erst danach Sensorzugriff |

Der Devicetree legt das Gerät fest; device_is_ready() bestätigt den nutzbaren Laufzeitzustand.

# Messwert erfassen: Fetch → Get

## Sample aktualisieren, Channel lesen, Wert prüfen

| 01 | 02 | 03 | 04 | 05 |
|---|---|---|---|---|
| **Fetch** | **Rückgabe** | **Get** | **Convert** | **Validate** |
| sensor_sample_fetch(dev) | 0 = Sample verfügbar | sensor_channel_get(...) | sensor_value_to_double() | Einheit, Bereich, Alter |

Fetch aktualisiert den Sensorzustand; Get liest einen konkreten Kanal daraus.

# struct sensor_value korrekt einordnen

**DARSTELLUNG**

```
val1 + val2 / 1 000 000 → physikalischer
Wert
```

## 01 Aufteilung

val1 enthält den Ganzzahlanteil, val2 die Millionstel.

## 02 Konvertierung

sensor_value_to_double() liefert eine bequeme Rechengröße.

## 03 Anwendungsmodell

Einheit normalisieren und Treiberformat nicht weiterreichen.

Die Anwendung übernimmt Bedeutung und Einheit – nicht die interne Sensordarstellung.

# Channels und Fähigkeiten

```
SENSOR-CHANNELS
SENSOR_CHAN_ACCEL_X / Y / Z
SENSOR_CHAN_ACCEL_XYZ
```

| | |
|---|---|
| **EINZELACHSE** | liefert genau eine Komponente |
| **VEKTORKANAL** | liefert alle Achsen, sofern vom Treiber unterstützt |
| **NICHT UNTERSTÜTZT** | -ENOTSUP beschreibt eine fehlende Fähigkeit |
| **AUFRUFER-VERTRAG** | nur benötigte Channels anfragen und Rückgabe prüfen |

Channel-Unterstützung ist eine Treibereigenschaft – kein allgemeines Sensorversprechen.

# Fehler richtig klassifizieren

CHANNEL

## Nicht unterstützt

-ENOTSUP

Alternative wählen

Fähigkeit dokumentieren

BUS

## Zugriff fehlgeschlagen

z. B. -EIO

Bus und Emulator prüfen

Fehler weitergeben

DATEN

## Ungültig oder unplausibel

Einheit prüfen

Bereich prüfen

Datenalter prüfen

Rückgabewert und Messkontext bestimmen gemeinsam die Reaktion der Anwendung.

# Emulator als kontrollierte Test-Schnittstelle

| | | |
|---|---|---|
| 01 | `Stimulus` | bekannte Beschleunigung reproduzierbar vorgeben |
| 02 | `Beobachtung` | denselben Treiber- und Sensor-API-Pfad verwenden |
| 03 | `Fehlerpfad` | nicht unterstützten Channel oder Busfehler gezielt auslösen |

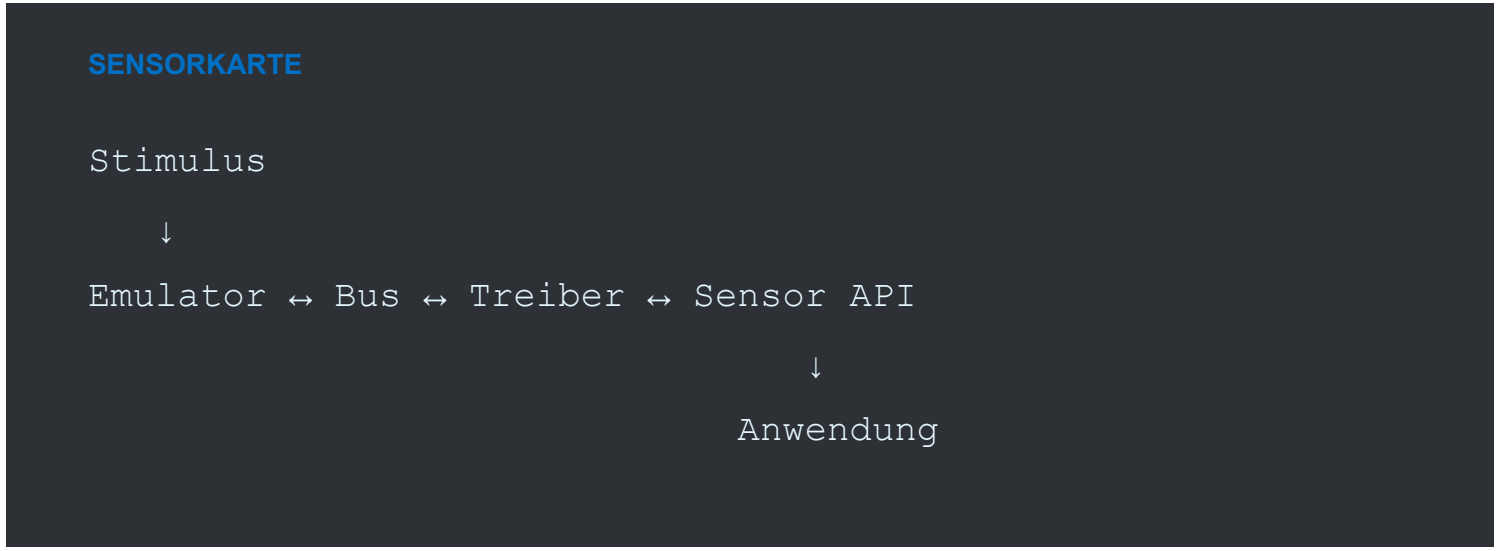
Deterministisch und wiederholbar bedeutet nicht physikalisch genau.

# Was Simulation belegt – und was nicht

| | | |
|---|---|---|
| 01 | **API-PFAD** | Devicetree, Binding, Treiber und API-Nutzung |
| 02 | **FACHLOGIK** | Konvertierung, Fehlerbehandlung und Datenfluss |
| 03 | **HARDWAREGRENZE** | keine Elektrik, Bus-Timings, Energie oder Sensorgenauigkeit |

Simulation prüft Softwareverhalten – reale Hardware bleibt ein eigener Nachweis.

# Lernkontrolle: Kette und Fehlerort




**01** **Erklären**

Weg eines Beschleunigungswertes

**02** **Zuordnen**

Fehler der verursachenden Ebene

Ergebnis: Sensorpfad und Fehlergrenzen werden ohne Lücken erklärt.